TRINA: Robust Graph Learning for Node Classification Against Trilateral Noise

Anonymous Author(s)

Abstract

Graph Neural Networks (GNNs) are powerful tools for learning from graph data, but they are highly vulnerable to real-world noise that can affect node features, graph structure, and labels. Existing methods typically address only one or two types of noise and often rely on the assumption that the remaining components are clean. However, when noise exists simultaneously in features, structure, and labels, such assumptions lead to interconnected corruption and unreliable supervision. Meanwhile, ground-truth graph data scales rapidly, requiring robust learning approaches to balance scalability, efficiency and performance. In this paper, we propose TRINA, a unified framework that jointly mitigates feature, structure, and label noise without relying on clean signals. TRINA employs a mutual structure-feature correction module to iteratively denoise both structure and feature, and uses a multi-teacher ensemble to generate robust and purified supervision labels. To enable its extension to large-scale graphs, TRINA leverages graph partitioning and adversarial learning. Experiments on real-world datasets demonstrate that TRINA consistently surpasses baseline models. For instance, with a 30% uniform noise rate, TRINA's accuracy is at least 7.85% higher than the current SOTA baselines on the Cora dataset, and it achieves a 5.04% accuracy improvement on ogbn-arxiv.

CCS Concepts

• **Information systems** → **Data cleaning**; • **Computing methodologies** → *Neural networks*; Learning latent representations.

Keywords

Trilateral noise, GNN, Robust, Large graphs

ACM Reference Format:

Anonymous Author(s). 2026. TRINA: Robust Graph Learning for Node Classification Against Trilateral Noise. In . ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Graph data has proven highly valuable across diverse domains, including social networks [8], financial transaction networks [33], and traffic networks [38]. Graph Neural Networks (GNNs) [36] are specifically designed to encode this relational information, typically relying on neighbor aggregation for node updates and backpropagation for label supervision. However, real-world graph data is frequently corrupted by various forms of noise. As Figure 1a illustrates, a malicious user like V3 might fabricate identity information, introducing *feature noise*. Alternatively, V3 could proactively follow legitimate users such as V2 and V5 to forge social relationships, causing *structural noise* within the network. Furthermore, through human annotation errors, V3 may be mislabeled (e.g., from Class A to Class B), leading to *label noise*. Each noise type disrupts graph learning in distinct ways. Feature noise corrupts a node's own representation (e.g., V3's features), while structural noise misguides neighborhood aggregation by enabling misleading information flow (e.g., from V3 to V2). Label noise compromises supervision, propagating incorrect signals during backpropagation and degrading the overall learning process.



(a) Trilateral noise **(b) Solution methods**

Figure 1: Examples of trilateral noise and corresponding solution methods.

Existing studies typically address individual noises separately, as summarized in Figure 1b. NRGNN [4] and RTGNN [28] employ graph augmentation and pseudo-labeling to mitigate noisy label propagation. RS-GNN [5], Pro-GNN [14] and STABLE [21] specifically target structural noise in graphs. Mid-GCN [11] designs novel convolution operations to handle structure noise. AirGNN [25] and NOSMOG [32] purify features through graph structure. However, all these methods address a single noise type. In this paper, we tackle a more general and complex scenario of *trilateral noise* in graph data, in which the graphs suffer from feature, structural, and label noise at the same time.

Recent research has started to tackle the problem of robust learning in the presence of multiple types of noise. For instance, RNCGLN [44] reduces structural noise using multi-hop attention-weighted edge refinement and addresses label noise through confidence-based pseudo-labeling. Similarly, RGIB [43] handles structural and label noise using two implementations derived from the graph information bottleneck [34]. However, a common limitation among these approaches is their tendency to process different noise types through separate modules.

Dealing with trilateral noise (feature, structural, and label noise) presents three critical challenges: (1) Interconnected corruption in message passing. Feature and structural noise can introduce corrupted message passing and create error-prone dependencies between them. For instance, methods that attempt to rectify structures using features, or rectify labels using both structure and features,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

may generate incorrect edges or labels because the underlying information sources are themselves unreliable. (2) Misleading supervision and optimization. Label noise can provide misleading supervision, adversely affecting the backpropagation process. Most methods designed for correcting structural or feature noise require label supervision. When labels are incorrect, they offer flawed guidance, hindering effective optimization and leading to suboptimal graph representations. These two challenges compound each other, making learning on graphs with trilateral noise significantly more difficult. (3) Scalability under joint denoising. Real-world graph datasets are growing rapidly. Multi-teacher training and iterative structure-feature correction introduce extra memory overhead and computational burden. Therefore, practical robust learners are required to deliver competitive performance on large-scale graphs.

PRINGLE [13] attempts to address the problem of trilateral noise using a joint modeling approach. It assumes that both structural and label noise originate solely from feature noise. While this assumption simplifies the problem, it oversimplifies the complex interactions among different types of noise present in real-world settings. For example, label and structural noise may arise independently from human annotation errors and data collection faults, rather than solely from feature corruption.

To address the above challenges, we propose **TRINA** (Trilateral noise-Robust graph learnIng for Node clAssification), a novel framework that jointly mitigates feature, structure, and label noise. To the best of our knowledge, TRINA is the first to tackle all three types of noise simultaneously without relying on oversimplified assumptions about their interactions. The core intuition behind TRINA is that, in a noise-free setting, node features, graph structure, and labels should exhibit mutual consistency. TRINA leverages this principle to guide both denoising and learning under noisy conditions. To prevent the propagation of corrupted signals during message passing, TRINA introduces a Structure-Feature Correction (SFC) module, which iteratively refines node features and graph structure based on their mutual dependencies, without assuming either modality to be clean. To address misleading supervision and optimization, TRINA adopts a two-stage learning paradigm. In the first stage, multiple teacher models produce soft-label distributions and consensus pseudo-labels, reducing reliance on any single noisy signal. In the second stage, a student model is trained using these cleaner labels, enabling robust learning without noise reinforcement. To scale TRINA to large graphs, TRINA incorporates Metis graph partitioning [18] and adversarial subgraph embedding alignment [6], thus making it possible for each partition can be trained under limited memory while embeddings remain consistent across partitions. Our main contributions are as follows:

- We present the first framework to jointly address feature, structure, and label noise without relying on error-prone dependencies or strong assumptions about the noise sources.
- We propose a mutual structure-feature correction module that performs iterative, cross-modal denoising of graph topology and node features.
- We design a multi-teacher, single-student architecture that produces robust pseudo-labels and soft-label distributions, enabling reliable supervision even under heavy noise.
- We extend TRINA to large graphs via Metis partitioning and adversarial subgraph embedding alignment, enabling scalable training while preserving trilateral-noise robustness.

2 Related Work

2.1 Single-source noise

2.1.1 Feature noise. Existing methods for correcting noisy node features typically rely on clean structural information to guide denoising. For example, NOSMOG [32] addresses the misalignment by complementing node content with position features to capture the graph structural information, and designs an innovative representational similarity distillation strategy. AirGNN [25] designs a message passing scheme with a homophily assumption on graph structure data and residual connection. Besides, GTrans [15] proposes a graph transformation framework that adapts and refines graph data at test time to achieve better performance. GCORNs [1] uses the weight matrix to handle feature noise.

2.1.2 Structure noise. Structural noise correction typically involves refining the graph topology while leveraging node features and various learning strategies. RSGNN [5] learns to connect feature-similar nodes, and STABLE [21] builds a denoised kNN graph. Mid-GCN [11] filters out noise using mid-frequency components, whereas EvenNet [20] improves robustness through modified message passing. Other methods adjust edge reliability via weighting [2, 16], edge pruning [31], or contrastive learning [45]. Recent work also explores using large language models to edit noisy graph structures [7].

2.1.3 Label Noise. Label noise is typically addressed by enhancing supervision through pseudo-labeling, clean sample selection, or structural refinement. Recent methods for graph label noise include contrastive learning [23, 26, 27, 40], sample scoring for clean label selection [4, 24, 28, 35], singular value decomposition [39], and subgraph-based approaches [37].

However, all of the above methods are designed under the assumption of single-source noise and are not directly applicable in scenarios of trilateral noise.

2.2 Multi-sources noise

Only a few existing methods explicitly address scenarios involving multi-source noise in graph data. RNCGLN [44] targets structural and label noise by leveraging multi-hop edge information with an attention mechanism for edge reweighting, followed by pseudo-labeling to mitigate label noise. SRGNN [41] addresses structural and label noise independently, refining graph structure through node features and using pseudo-labels for robust supervision. RGIB [43] tackles edge-level structural and label noise via graph augmentation and clean signal extraction, grounded in the graph information bottleneck principle. However, these methods still handle each noise type separately.

PRINGLE [13] stands out as a unique method designed to handle trilateral noise, which encompasses corruption in node features, graph structure, and labels. It introduces a feature-driven generative noise hypothesis (FDGN), modeling both structural and label noise as originating from feature noise. This causal perspective

then guides a joint denoising process. However, a notable limitation of PRINGLE is its reliance on strong assumptions that both structural and label noise originate solely from feature noise, which can restrict its applicability across diverse real-world scenarios.

2.3 Large Graph Training

Scaling GNNs to large graphs typically relies on graph partitioning or sampling to reduce per-step memory. Metis multilevel partitioning [18] minimizes edge cuts under balance constraints, and Cluster-GCN [3] exploits such partitions to restrict neighborhood aggregation within dense clusters. Complementary to partitioning, adversarial learning with a gradient reversal layer [6] is widely used to align feature distributions across domains. Nevertheless, existing scalable training pipelines rarely address feature, structure, and label noise jointly. TRINA couples trilateral-noise robust learning with Metis partitioning and adversarial subgraph alignment to remain effective at large scale.

3 Problem Definition

Definition 1 (Graph). *A graph $\mathcal{G}$ is defined as a tuple $\langle \mathcal{V}, \mathcal{E}, \mathcal{X} \rangle$, where $\mathcal{V} = \{v_1, \ldots, v_N\}$ is the set of nodes, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges, and $\mathcal{X} \in \mathbb{R}^{N \times F}$ represents the node feature matrix, with F being the number of features. Each node v_i is associated with a feature vector $\mathcal{X}_i \in \mathbb{R}^F$ and a one-hot encoded label $y_i \in \{0,1\}^C$, where C is the number of classes. The observed graph structure is represented by an adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, where $\mathbf{A}_{ij} = 1$ if there exists an edge between v_i and v_j, and $\mathbf{A}_{ij} = 0$ otherwise.*

Definition 2 (Trilateral Noise Graph). *A trilateral noise graph is one that is simultaneously corrupted by noise in its labels, structure, features: (1) Label Noise: The provided labels $\mathcal{Y}$ for $\mathcal{V}_L$ are corrupted versions of the true labels. (2) Structure Noise: The observed adjacency matrix $\mathcal{A}$ deviates from the true graph structure. (3) Feature Noise: The observed node features $\mathcal{X}$ are noisy variants of the true features.*

Definition 3 (Node Classification). *Given a graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E}, \mathcal{X} \rangle$, the node set is divided into a labeled subset $\mathcal{V}_L$ and an unlabeled subset $\mathcal{V}_U = \mathcal{V} \setminus \mathcal{V}_L$. The objective is to train a model using the labeled nodes in $\mathcal{V}_L$, in order to accurately predict the true labels of nodes in $\mathcal{V}_U$.*

Our objective is to solve the node classification problem in a trilateral noise graph:

4 Methodology

4.1 Framework Overview

As mentioned in the introduction, addressing noise in features, graph structure, and labels presents two primary challenges: interconnected corruption and misleading supervision. We address corruption in both features and graph structure using the Structure-Feature Correction (SFC) module, which iteratively refines them by modeling their mutual dependencies. In each training epoch, the SFC module corrects the graph structure and features by referencing both the original and the previous epoch's graph. This prevents the model from straying too far from the initial data while still progressively cleaning the graph. Consequently, each epoch yields a cleaner, more reliable graph representation. Building on SFC, we propose a teacher-student architecture trained in two stages to address the misleading supervision issue. The first stage leverages an ensemble of teacher models to distill robust, consensus-based pseudo-labels, effectively filtering the initial label noise. In the second stage, a student model learns from this purified supervisory label. The design is motivated by the observation that, in clean settings, features, structure, and labels tend to be mutually consistent. By encouraging this consistency during training, TRINA progressively aligns the learning process toward a noise-free solution. The overall architecture is illustrated in Figure 2. On medium-sized graphs we train TRINA over the full graph; on large graphs we enable the Metis partitioning and adversarial alignment extension.

In the following subsection, we first present the structure of Structure-Feature Correction module, which aims to address the noises in structure and feature. Then, we introduce the Teacher-Student architecture, aiming to address the noises in the label. Finally, we describe how TRINA scales to large graphs.

4.2 Structure-Feature Correction (SFC) Module

The Structure and Feature Correction (SFC) module serves as a fundamental building block for denoising both graph structure and node features. It consists of two parallel submodules: the Structural Cleaning (SC) submodule and the Feature Cleaning (FC) submodule, which are responsible for refining the adjacency matrix and denoising the node features, respectively.

In both the teacher and student models, the SFC module takes four inputs: the original adjacency matrix and node feature matrix, along with a purified adjacency matrix and purified node feature matrix from the previous training epoch. Based on these inputs, the SFC module produces updated (i.e., more purified) versions of the adjacency matrix and node features, which are then used in the subsequent epoch to progressively enhance the graph structure and node features. For the first training epoch, since no purified information from a previous epoch is available, we initialize the purified adjacency matrix using a K-Nearest Neighbor (KNN) graph constructed from the original node features, and set the purified feature matrix to be identical to the original feature matrix. We next elaborate on the SC submodule and FC submudule in details.

4.2.1 Structural Cleaning (SC). The Structural Cleaning (SC) submodule progressively refines a graph's structure throughout the training process. It addresses potential inaccuracies in the initial K-Nearest Neighbors (KNN) graph, which is constructed from noisy feature data. To correct this, SC iteratively improves the graph by using progressively cleaner node representations to identify and remove erroneous connections. To prevent going too far and removing correct edges, we've designed a structure loss. Simultaneously, a noise-robust loss offers flexible supervision, preventing the SC submodule's structural refinements from deviating excessively from the available label information.

Specifically, at each epoch i, SC takes three inputs: the original adjacency matrix $\mathbf{A}_o$, node features matrix from the previous epoch $\mathbf{X}_t^{i-1}$ and adjacency matrix from the previous epoch $\mathbf{A}_t^{i-1}$.

First, the original adjacency matrix $\mathbf{A}_o$ and the updated node features from the previous epoch $\mathbf{X}_t^{i-1}$ are fed into a GCN to compute

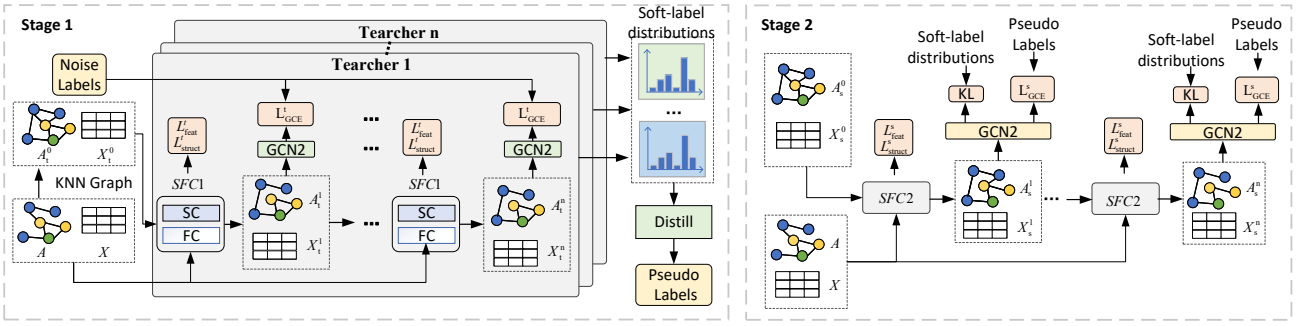


Figure 2: The two-stage architecture: (1) Stage 1 trains teacher models; (2) Stage 2 trains student model for final classification.

node representations:

$$\mathbf{H} = \mathrm{GCN1}(\mathbf{A}_o, \mathbf{X}_t^{i-1}). \tag{1}$$

Next, to adaptively fuse the initial and previous adjacency matrices and prevent the graph deviating excessively from its original topology, a learnable binary mask $\mathbf{W}_1 \in \{0,1\}^{n\times n}$ is introduced:

$$\mathbf{A}_{\text{fused}} = \mathbf{W}_1 \odot \mathbf{1}_{\mathbf{A}_t^0 \neq 0} + (\mathbf{1} - \mathbf{W}_1) \odot \mathbf{1}_{\mathbf{A}_t^{i-1}}, \tag{2}$$

where $\odot$ denotes element-wise multiplication, $\mathbf{1}$ is an all-ones matrix, and $\mathbf{1}_{A\neq 0}$ returns 1 if the condition is non-zero and 0 otherwise. For each candidate edge (v_i, v_j) in $\mathbf{A}_{\text{fused}}$ (i.e., $\mathbf{A}_{\text{fused}}[i][j] \neq 0$), an edge confidence score is calculated using node embeddings:

$$p_{j,k} = \sigma(sim(\mathbf{H}[j], \mathbf{H}[k])), \tag{3}$$

where $\mathrm{sim}(\cdot,\cdot)$ denotes cosine similarity and σ is the ReLU activation function.

The refined adjacency matrix $\mathbf{A}_t^i$ is then constructed by retaining only those edges whose confidence score exceeds a predefined threshold τ:

$$\mathbf{A}_t^i[j][k] = \begin{cases} p_{j,k} & \text{if } \mathbf{A}_{\text{fused}}[j][k] \neq 0 \text{ and } p_{j,k} > \tau, \\ 0 & \text{otherwise.} \end{cases} \tag{4}$$

The resulting structure $\mathbf{A}_t^i$ serves as the input adjacency matrix for the subsequent epoch $i+1$.

To ensure that the model retains essential connectivity patterns while filtering out noisy or irrelevant connections, we introduce a structure reconstruction loss:

$$\mathcal{L}_{\text{struct}}^t = \frac{N}{|\mathcal{E}^+| + |\mathcal{E}^-|}\left[\sum_{(v_j,v_k)\in\mathcal{E}^+} \left(p_{j,k}^{\text{reg}} - 1\right)^2 + \sum_{(v_j,v_k)\in\mathcal{E}^-} \left(p_{j,k} - 0\right)^2\right], \tag{5}$$

where N is the number of nodes, and $\mathcal{E}^+$ and $\mathcal{E}^-$ represent the sets of positive (existing) and randomly sampled negative (non-existing) edges, respectively. The regularized prediction p_{jk}^{reg} is computed as:

$$p_{j,k}^{\text{reg}} = \theta \cdot \mathrm{sim}(\mathcal{X}[j], \mathcal{X}[k]) + (1-\theta)\cdot p_{j,k}, \tag{6}$$

where $p_{j,k}^{\text{reg}}$ is a weighted average of the original predicted probability and the cosine similarity of the nodes' original features, $\mathrm{sim}(\mathcal{X}[j], \mathcal{X}[k])$. The hyperparameter $\theta \in [0,1]$ controls the balance between these two components. This regularization is applied to positive edges to reinforce their validity using feature similarity, thereby mitigating the impact of noisy connections that might exist in the ground-truth graph. For negative edges, we directly use $p_{j,k}$ since they are randomly sampled and assumed to be false.

4.2.2 Feature Cleaning. The Feature Cleaning (FC) submodule aims to iteratively suppress feature noise during training. Because the initial input features may be corrupted, FC reconstructs each node's representation using its neighborhood information to identify and correct noisy features. At each epoch, nodes whose features significantly deviate from their reconstructed counterparts, are considered noisy and corrected accordingly. The underlying intuition is that normal nodes are usually similar to their neighbors, while noisy nodes tend to deviate from this local consistency. To prevent excessive modification of the features, a penalized feature reconstruction error loss is introduced to limit the degree of change. Additionally, a noise robust loss provides adaptive supervision.

At each epoch i, the module takes three inputs: the features from the previous epoch $\mathbf{X}_t^{i-1}$, the original input adjacency matrix $\mathbf{A}_o$ the original input features $\mathbf{X}_o$. To balance the preservation of raw input information and the refinement from previous epochs, the module first fuses $\mathbf{X}_t^{i-1}$ and $\mathbf{X}_o$ using an adaptive interpolation strategy. Specifically, a learnable weight matrix $\mathbf{W}_2 \in [0,1]^{n\times d}$ is used to control the contribution of each feature dimension:

$$\mathbf{X}_{\text{fused}} = \mathbf{W}_2 \odot \mathbf{X}_t^{i-1} + (\mathbf{1} - \mathbf{W}_2) \odot \mathbf{X}_o, \tag{7}$$

where $\odot$ denotes the Hadamard product and $\mathbf{1}$ represents an all-ones matrix. The fused features $\mathbf{X}_{\text{fused}}$ are then passed into GraphMAE [9], a graph masked autoencoder that leverages the original adjacency matrix $\mathbf{A}_o$ to perform structure-aware reconstruction.

$$\mathbf{X}'_{\text{fused}} = \mathrm{GraphMAE}(\mathbf{A}_o, \mathbf{X}_{\text{fused}}). \tag{8}$$

To prevent trivial identity mapping and encourage meaningful learning, GraphMAE randomly selects a subset of nodes $\tilde{\mathcal{V}} \subset \mathcal{V}$ and replaces their features with a learnable mask token $x_{[M]} \in \mathbb{R}^F$:

$$\tilde{\mathbf{X}}_{\text{fused}}[i] = \begin{cases} x_{[M]} & v_i \in \tilde{V}, \\ \mathbf{X}_{\text{fused}}[i] & v_i \notin \tilde{V}. \end{cases} \tag{9}$$

The masked feature matrix $\tilde{\mathbf{X}}_{\text{fused}}$ is then input to the graph autoencoder, producing reconstructed features $\mathbf{X}'_{\text{fused}}$. Although this masking strategy promotes robust feature learning, it can create a mismatch between training and inference since masked tokens do not appear during deployment.

The FC module adopts the Scaled Cosine Error (SCE) loss [9], which penalizes reconstruction errors with an exponential scaling factor $\gamma > 1$:

$$\mathcal{L}^t_{\text{feat}} = \frac{1}{|\tilde{\mathcal{V}}|} \sum_{v_i \in \tilde{\mathcal{V}}} \left(1 - \frac{\mathbf{X}'_{\text{fused}}[i] \cdot \mathbf{X}_o[i]}{|\mathbf{X}'_{\text{fused}}[i]| \cdot |\mathbf{X}_o[i]|}\right)^{\gamma}. \tag{10}$$

This loss accentuates difficult reconstruction cases and helps the model focus on noisy or corrupted features for subsequent correction.

To identify noisy nodes, the module calculates the element-wise reconstruction error using the L1 norm:

$$\Delta = |\mathbf{X}' - \mathbf{X}_o|. \tag{11}$$

The top-k nodes with the highest error scores in Δ are selected as the noisy set, $\mathcal{V}_{\text{noisy}}$. The assumption is that nodes with a large difference between their original and reconstructed features are likely to be corrupted or uninformative.

For each node $v_k \in \mathcal{V}_{\text{noisy}}$, its feature vector is corrected by replacing it with the average of its neighbors' feature vectors from the previous epoch. This can be expressed as:

$$\mathbf{X}^i_t[k] = \frac{1}{|\mathcal{N}(v_k)|} \sum_{v_j \in \mathcal{N}(v_k)} \mathbf{X}^{i-1}_t[j], \tag{12}$$

where $\mathcal{N}(v)$ denotes the set of neighbors of node v in $\mathbf{A}_o$. This updated feature matrix is then used as input for the next epoch, allowing the model to progressively denoise features over time.

4.3 Training the Teacher-Student Architecture

To address the label noise, we employs a multi-teacher single-student architecture [12], drawing inspiration from effective knowledge distillation techniques [22].

4.3.1 Training of teacher models. The teacher models are trained independently with different initializations to encourage diversity. Specifically, in a teacher model, the purified node features $\mathbf{X}^i_t$ and adjacency matrix $\mathbf{A}^i_t$ at epoch i are fed into a GNN-based predictor to generate label predictions $\hat{\mathbf{Y}}$. This predictor, an implementation of a weighted Graph Convolutional Network (GCN) [19], employs layer-wise propagation:

$$\mathbf{H}^{(i)} = \text{GCN2}(\mathbf{A}^i_t, \mathbf{X}^i_t). \tag{13}$$

Subsequently, a softmax output layer converts these representations into class probability distributions:

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{H}^{(i)}), \tag{14}$$

where $\hat{\mathbf{Y}} \in \mathbb{R}^{|\mathcal{V}| \times C}$, where C is the number of classes, and each entry $\hat{y}_{v,c}$ denotes the predicted probability of node v belonging to class c.

To enhance robustness against label noise, we adopt the Generalized Cross Entropy (GCE) loss [42], which bridges the noise tolerance of mean absolute error and the optimization efficiency of standard cross entropy. For multi-class classification with a one-hot ground-truth vector $y_v = \{y_{v,1}, \ldots, y_{v,C}\}$, the GCE loss is defined as:

$$\mathcal{L}^t_{\text{GCE}} = \frac{1}{q} \sum_{v \in \mathcal{V}} \sum_{c=1}^{C} y_{v,c} \left(1 - \hat{y}^q_{v,c}\right), \tag{15}$$

where $q \in (0, 1]$ controls the trade-off between robustness and sensitivity, y_c indicates whether class c is the true label of node v.

The overall training loss for a teacher model at each epoch is a composite of three terms: structure, features, and label supervision:

$$\mathcal{L}^t_{total} = \lambda_1 \mathcal{L}^t_{\text{GCE}} + \lambda_2 \mathcal{L}^t_{\text{struct}} + \lambda_3 \mathcal{L}^t_{\text{feat}}. \tag{16}$$

4.3.2 Training of student model. Once all teacher models have been trained, they potentially specialize in different aspects of noise patterns. We next train the student model to synthesize more comprehensive knowledge by learning from multiple teachers. This ultimately leads to better generalization than any single teacher could achieve alone.

Let $\hat{\mathbf{Y}}_k$ denote the predicted probability distributions from the k-th teacher. To transfer soft knowledge, the student minimizes the KL divergence between its predictions and those from each teacher:

$$\mathcal{L}^s_{\text{KLD}} = \sum_{k=1}^{K} \sum_{v \in \mathcal{V}} \text{KL}\left(\hat{Y}^v_k \parallel \hat{Y}^v_{\text{student}}\right), \tag{17}$$

where $\hat{Y}^v_k$ and $\hat{Y}^v_{\text{student}}$ denote the predicted class probability distributions for node v from the k-th teacher and the student model, respectively.

Beyond soft label alignment, we further purify supervision by extracting high-confidence samples. A node v is deemed clean by the k-th teacher if the highest predicted probability exceeds a confidence threshold τ:

$$C_k = \left\{v \in \mathcal{V} \mid \max(\hat{Y}^v_k) > \tau\right\}. \tag{18}$$

Here, $\hat{Y}^v_k$ represents the predicted probability distribution of node v by the k-th teacher. Then, the set of high-confidence samples from all teachers is:

$$C = \bigcap_{k=1}^{n} C_k. \tag{19}$$

Nodes in the set C serve as high-quality supervision sources, guiding the student model to jointly mitigate both structural and feature-level noise. The hard label loss is shown as:

$$\mathcal{L}^s_{\text{GCE}} = \frac{1}{q} \sum_{v \in C} \sum_{c=1}^{C} y_{v,c} \left(1 - \hat{y}^q_{v,c}\right). \tag{20}$$

By integrating soft label distillation and hard label correction, the student model benefits from both generalized knowledge transfer and noise-resistant supervision. The total training objective is formulated as a weighted combination of four loss components:

$$\mathcal{L}^s_{\text{total}} = \lambda_1 \mathcal{L}^s_{\text{GCE}} + \lambda_2 \mathcal{L}^s_{\text{struct}} + \lambda_3 \mathcal{L}^s_{\text{feat}} + \lambda_4 \mathcal{L}^s_{\text{KLD}}, \tag{21}$$

where $\mathcal{L}^s_{\text{struct}}$ and $\mathcal{L}^s_{\text{feat}}$ are computed in the same way as their teacher counterparts, i.e., $\mathcal{L}^t_{\text{struct}}$ and $\mathcal{L}^t_{\text{feat}}$. Note that the KLD term is omitted during teacher training (i.e., $\lambda_4 = 0$), while the student model optimizes all four terms. This multi-objective learning strategy preserves graph structure, suppresses feature and label noise, and effectively distills complementary knowledge from the teacher ensemble for enhanced robustness.

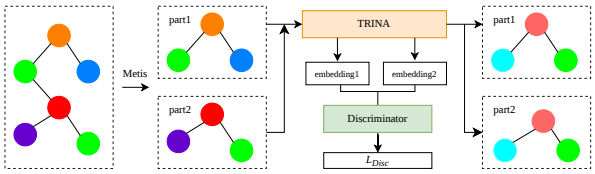


Figure 3: Large-scale extension of TRINA: Metis partitions the graph and adversarial discriminator aligns embeddings.

4.4 Scaling to Large Graphs

Applying TRINA directly to large graphs will face multiple realistic constraints: adjacency matrix surge with the number of nodes, SFC iterative denoising and multi-teacher integration bring high computational overhead. Ordinary subgraph sampling will lose neighborhood information and destroy the edge confidence estimation of the SC module. Thus, we extend the model based on Metis [18] graph partitioning combined with adversarial subgraph embedding alignment [6]. Metis partition can balance the size of subgraphs, reducing edge loss across partitions, and is suitable for splitting the original graph. Adversarial alignment specifically alleviates the distribution shift of subgraph representation caused by partitioning, ensuring the consistency of embedding space after splitting training, and completely retains all three-party noise denoising capabilities of TRINA. Details are illustrated in Figure 3.

4.4.1 Metis Graph Partitioning. We partition the original graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{X})$ into P subgraphs $\{G_k = (A_k, X_k)\}_{k=1}^{P}$ using Metis multilevel partitioning [18]. Let $\Pi = \{\mathcal{V}_1, \ldots, \mathcal{V}_P\}$ denote a partition of the node set. Metis approximately minimizes the cross-partition edge cut subject to a load-balance constraint that keeps partition sizes comparable:

$$\mathrm{cut}(\Pi) = \sum_{1 \le a < b \le P} \sum_{u \in \mathcal{V}_a,\, v \in \mathcal{V}_b} \mathbf{A}_{uv}, \quad |\mathcal{V}_k| \approx \frac{|\mathcal{V}|}{P}. \tag{22}$$

Compared with random cuts, Metis better preserves local community structure that message passing relies on. Each subgraph then runs the same SFC module and teacher–student training as in the full-graph setting.

4.4.2 Adversarial Subgraph Embedding Alignment. Different partitions may exhibit distinct topology densities and community statistics, inducing subgraph-specific embedding shifts that hurt cross-partition generalization. Viewing each partition as a domain, we introduce a multilayer perceptron (MLP) discriminator D_ϕ and train the GNN encoder f_θ adversarially so that embeddings become partition-invariant [6].

Let $H_k = f_\theta(A_k, X_k)$ denote the embeddings on partition k, and let h_i be the embedding of node v_i. The discriminator outputs a categorical distribution over the P partitions,

$$p_i = D_\phi(h_i) = \mathrm{softmax}\big(\mathrm{MLP}(h_i)\big) \in \mathbb{R}^P, \tag{23}$$

where the m-th entry $p_i^{(m)}$ is the predicted probability that v_i belongs to partition m. Given the true partition label $y_i \in \{0,1\}^P$, the discriminator is trained by minimizing the multi-class cross-entropy:

$$\mathcal{L}_{\mathrm{disc}} = -\sum_{v_i \in \mathcal{V}} \sum_{m=1}^{P} y_i^{(m)} \log p_i^{(m)}. \tag{24}$$

A stronger discriminator indicates that embeddings still carry partition-specific bias. Conversely, if f_θ succeeds in aligning distributions across partitions, D_ϕ becomes harder to train.

Let $\mathcal{L}_{\mathrm{task}}$ denote the original TRINA training loss on each subgraph. We couple the task objective with adversarial alignment and optimize the two networks alternately:

$$\min_{\phi} \ \mathcal{L}_{\mathrm{disc}}, \qquad \min_{\theta} \ \mathcal{L}_{\mathrm{task}} - \lambda_a \mathcal{L}_{\mathrm{disc}}, \tag{25}$$

where λ_a balances task learning and partition invariance. In each training round, we first fix the encoder parameters θ and update only the discriminator parameters ϕ to better identify partition origins; we then fix ϕ and update θ through a gradient reversal layer (GRL), which flips the sign of $\partial \mathcal{L}_{\mathrm{disc}}/\partial\theta$ so that the encoder confuses D_ϕ.

4.5 Time Complexity.

The total training complexity of the teacher model and the student model per epoch is the sum of the computational costs of SFC and the subsequent GCN classifier. SFC involves two parallel submodules SC and FC. The complexity of SC is $O(|V| \cdot d^2 + (|E| + |E_{\mathrm{fused}}|) \cdot d)$, where $|V|$ is the number of nodes, $|E|$ is the number of edges in the original graph, $|E_{\mathrm{fused}}|$ is the number of edges in the fused graph, and d is the feature dimension. FC's complexity is determined by the GraphMAE component, leading to a cost of $O(|V| \cdot d^2 + |E| \cdot d)$. The GCN classifier's complexity is $O(|V| \cdot d^2 + |E_{\mathrm{purified}}| \cdot d)$, where $|E_{\mathrm{purified}}|$ is the number of edges in the purified graph for the current epoch. By assuming $|E|$, $|E_{\mathrm{fused}}|$ and $|E_{\mathrm{purified}}|$ are on the same order of magnitude, the complexity for training a teacher/student model per epoch on a full graph is $O(|V| \cdot d^2 + |E| \cdot d)$.

For large graphs with P Metis partitions (e.g., P=4 on ogbn-arxiv), each partition contains roughly $|V|/P$ nodes and $|E|/P$ edges. Training on a single partition therefore costs about $O((|V|/P) \cdot d^2 + (|E|/P) \cdot d)$ per epoch, reducing peak memory and per-step computation by a factor of approximately P. The adversarial discriminator adds only a lightweight MLP classification cost that is dominated by message passing. The overall training cost is the sum of teacher/student costs over the P partitions. During inference, the iterative SFC module is bypassed. The process consists of a single forward pass through the trained student GCN, with time complexity $O(|V| \cdot d^2 + |E_{\mathrm{final}}| \cdot d)$, where $|E_{\mathrm{final}}|$ is the number of edges in the final purified graph structure.

5 Experiments

In this section, we present experimental results on real-world datasets to demonstrate the effectiveness of our proposed framework. Specifically, we investigate the following research questions:

- **RQ1** How robust is the proposed TRINA framework to trilateral noise and varying noise levels?
- **RQ2** Is the proposed framework robust to different noise types?

Table 1: Statistics of datasets.

Dataset	# nodes	# edges	# features	# classes
Cora	2708	5429	1433	7
Citeseer	3312	4723	3703	6
Photo	7650	119,081	745	8
Pubmed	19,717	44,338	500	3
ogbn-arxiv	169,343	1,166,243	128	40

- **RQ3** Does each individual module of the model contribute to improved prediction accuracy?
- **RQ4** Does the model exhibit satisfactory computational efficiency?

5.1 Experimental Settings

5.1.1 Datasets. We evaluate the robustness of our proposed method on five datasets: three citation networks (Cora, Citeseer, and Pubmed) [29], one Amazon co-purchase network (Photo) [30], and one large-scale benchmark (ogbn-arxiv) [10]. Table 1 summarizes dataset statistics.

Following prior work [13, 28, 44], we assessed model robustness by injecting two common types of noise: uniform and pair noise. In addition, we introduced two novel noise types. (1) Feature-based noise: Injects noise into the k edges furthest from a node's feature representation. (2) Class confusion noise: Replaces a targeted node's features with those from samples of other categories.

We consider three distinct trilateral noise settings by varying the ratios of structure noise (gr), label noise (lr), and feature noise (fr). Specifically, we evaluate the model under $\text{gr}, \text{lr}, \text{fr} = 0, 0, 0$ (noise-free), $0.1, 0.1, 0.1$ (mild noise), and $0.3, 0.3, 0.3$ (severe noise). This design allows for an intuitive comparison across increasing levels of corruption.

5.1.2 Evaluation metrics. Our experiment employs two complementary metrics to assess model performance under noise: Accuracy and F1-Macro.

5.1.3 Baselines. The comparison methods include GCN [19], which does not address either graph noise or label noise; two multi-noise robust methods, RNCGLN [44] and PRINGLE [13]; one label-noise robust method, RTGNN [28]; and one graph-noise robust method, Mid-GCN [11]. Details are provided in the Appendix.

5.1.4 Implementation Details. Both the teacher and student SC modules adopt a single-layer GCN with 128 hidden units (denoted as GCN1) as the encoder. Their prediction models are implemented as two-layer GCNs with 128 hidden units (denoted as GCN2). The hyperparameter τ in Eq. 4 is set to 0.05 across all datasets. We fix the loss weights as $\lambda_1 = 1$, $\lambda_2 = 0.03$, $\lambda_3 = 1$, $\lambda_4 = 300$; for teacher models, the KLD term is disabled by setting $\lambda_4 = 0$. We perform hyperparameter search for the number of nearest neighbors K in k-nearest neighbor (KNN) graph from the set $\{50, 100, 200, 400, 600, 800\}$ based on the validation performance. The number of teacher models is fixed at 4. The model is trained for 2000 epochs using the Adam optimizer with a learning rate of 5×10^{-4} and a weight decay of 5×10^{-4}. To prevent overfitting, we apply dropout with a rate of 0.5, and adopt early stopping. All reported results are averaged over 4 independent runs. On medium-sized datasets we train TRINA on the full graph without partitioning or adversarial alignment. On ogbn-arxiv, we enable the large-graph extension: Metis partitions the graph into 4 subgraphs, the KNN neighborhood size is set to K=200, and the adversarial alignment weight is λ_a=0.1. KNN graph construction is accelerated by FAISS [17].

5.2 Performance Comparison (RQ1 and RQ2)

This section compares the performance of all methods across different levels and types of noise.

5.2.1 Performance on Different Levels of Trilateral Noise. Table 2 presents node classification results under uniform trilateral noise, demonstrating TRINA's state-of-the-art performance. GCN shows significant performance degradation as noise increases, while methods designed for single noise types such as RT-GNN and Mid-GCN exhibit some robustness but remain less effective than multi-noise approaches. Multi-noise methods like RNCGLN and PRINGLE generally perform well but occasionally underperform. For example, RNCGLN scored lower than GCN on CiteSeer and Photo at noise level $0.3, 0.3, 0.3$, and PRINGLE performed worse than Mid-GCN on Pubmed under the same noise conditions. In contrast, TRINA consistently achieves the highest or near highest accuracy and F1 macro scores. Notably, at noise level $0.3, 0.3, 0.3$, TRINA outperforms the second best method by 7.85 percent on Cora and by 8.6 percent on Pubmed. On the large-scale ogbn-arxiv benchmark, TRINA remains the strongest method across all noise levels. At noise level $0.3, 0.3, 0.3$, TRINA improves accuracy by 5.04 percentage points over the second-best baseline PRINGLE, indicating that Metis partitioning with adversarial alignment preserves TRINA's robustness at scale.

5.2.2 Performance on Different Noise Injection Methods. We further compare TRINA with baseline methods under three challenging noise settings: pair noise, feature-based noise, and class confusion noise. Table 3 reports the average performance and standard deviation over four independent runs. The results show that all methods experience performance declines under pair noise, but TRINA consistently outperforms others, especially at high noise levels on the Cora dataset. Under feature-based noise, performance drops across all methods on the Pubmed dataset, yet TRINA remains nearly the best in all cases and matches the top method PRINGLE on the noise-free Photo dataset. Class confusion noise causes the greatest disruption for all methods, but TRINA achieves the highest performance in most experiments and demonstrates comparable results to the best model on the Photo dataset. Detailed results at varying noise rates are provided in the Appendix.

5.3 Model Ablation Study (RQ3)

We conduct an ablation study to evaluate the contribution of each module in TRINA. To assess the effectiveness of the multi-teacher-single-student framework, we compare TRINA with a single-teacher variant, denoted as "TRINA_T". To verify the role of the GCE loss, we replace it with standard cross-entropy in a variant labeled "w/ CE." We also individually remove the Structure Cleaning (SC) and Feature Cleaning (FC) modules, denoted as "w/o SC" and "w/o FC", respectively. To examine the impact of feature iteration on structure refinement, the variant "w/ X_o" uses the original noisy features X_o

Table 2: Performance Comparison w.r.t uniform noise (Accuracy(%)±Std & F1 Macro Score)

Dataset	Metrics	{gr,lr,fr}	GCN	Mid-GCN	RT-GNN	PRINGLE	RNCGLN	TRINA
Cora	Accuracy	{0,0,0}	84.26±0.52	84.50±0.70	78.02±0.31	83.13±0.09	83.11±0.70	**85.19±0.86**
		{.1,.1,.1}	73.53±0.13	73.82±0.29	71.57±0.37	77.95±0.23	75.59±1.59	**81.33±0.32**
		{.3,.3,.3}	54.02±0.45	57.70±0.42	58.81±0.98	61.60±0.58	65.33±0.62	**73.18±1.00**
	F1 Score	{0,0,0}	0.83±0.00	0.83±0.00	0.69±0.00	0.82±0.00	0.81±0.01	**0.84±0.00**
		{.1,.1,.1}	0.72±0.00	0.72±0.00	0.64±0.01	0.77±0.00	0.73±0.02	**0.81±0.00**
		{.3,.3,.3}	0.50±0.01	0.53±0.02	0.29±0.22	0.60±0.01	0.63±0.00	**0.73±0.01**
CiteSeer	Accuracy	{0,0,0}	67.98±0.62	70.55±0.45	67.58±1.54	71.40±1.63	70.63±1.11	**72.38±0.89**
		{.1,.1,.1}	68.23±1.22	61.88±1.11	60.13±1.23	68.60±1.49	64.95±0.60	**68.88±0.93**
		{.3,.3,.3}	56.03±0.58	49.62±0.35	44.86±3.53	**58.80±0.14**	51.20±1.41	58.10±0.75
	F1 Score	{0,0,0}	0.65±0.01	0.67±0.00	0.66±0.01	**0.68±0.02**	0.67±0.01	**0.68±0.00**
		{.1,.1,.1}	0.60±0.01	0.60±0.01	0.61±0.00	0.66±0.02	0.62±0.01	**0.67±0.00**
		{.3,.3,.3}	0.46±0.01	0.45±0.01	0.45±0.02	0.54±0.00	0.48±0.02	**0.55±0.01**
Photo	Accuracy	{0,0,0}	92.36±0.13	81.96±0.06	88.96±0.25	**94.39±0.05**	87.16±0.62	93.77±0.06
		{.1,.1,.1}	88.40±0.39	75.49±0.25	85.28±0.90	**91.34±0.22**	80.02±0.19	**91.15±0.30**
		{.3,.3,.3}	84.70±0.30	57.01±0.28	77.45±1.01	86.72±0.15	59.91±0.36	**87.16±0.78**
	F1 Score	{0,0,0}	0.90±0.00	0.72±0.00	0.79±0.00	**0.93±0.00**	0.82±0.01	0.92±0.00
		{.1,.1,.1}	0.86±0.01	0.65±0.01	0.76±0.00	**0.89±0.01**	0.73±0.01	**0.89±0.00**
		{.3,.3,.3}	**0.79±0.03**	0.37±0.01	0.60±0.02	0.83±0.00	0.47±0.00	0.78±0.01
Pubmed	Accuracy	{0,0,0}	76.63±0.55	77.98±0.49	78.06±1.23	75.20±0.71	76.18±0.61	**78.78±0.43**
		{.1,.1,.1}	71.73±0.33	72.00±0.35	71.66±0.95	68.43±0.74	70.93±0.80	**76.10±0.24**
		{.3,.3,.3}	52.75±0.57	55.15±0.53	50.79±2.43	51.90±0.51	56.80±0.96	**65.40±0.51**
	F1 Score	{0,0,0}	0.76±0.01	**0.78±0.00**	0.77±0.01	0.75±0.00	0.76±0.01	**0.78±0.01**
		{.1,.1,.1}	0.70±0.00	0.70±0.00	0.71±0.01	0.68±0.01	0.70±0.01	**0.75±0.01**
		{.3,.3,.3}	0.50±0.00	0.54±0.00	0.50±0.02	0.49±0.00	0.55±0.01	**0.63±0.00**
ogbn-arxiv	Accuracy	{0,0,0}	55.79±0.13	36.59±0.09	53.61±0.06	59.70±1.92	OOM	**60.30±0.31**
		{.1,.1,.1}	41.46±0.05	30.97±0.17	43.16±0.11	53.07±1.57	OOM	**54.55±0.22**
		{.3,.3,.3}	31.22±0.11	30.12±0.06	31.96±0.11	42.65±1.20	OOM	**47.69±0.19**
	F1 Score	{0,0,0}	0.17±0.00	0.03±0.00	0.16±0.00	0.23±0.01	OOM	**0.24±0.01**
		{.1,.1,.1}	0.10±0.00	0.02±0.00	0.12±0.00	0.20±0.01	OOM	**0.22±0.01**
		{.3,.3,.3}	0.02±0.00	0.01±0.00	0.05±0.00	0.10±0.01	OOM	**0.15±0.00**

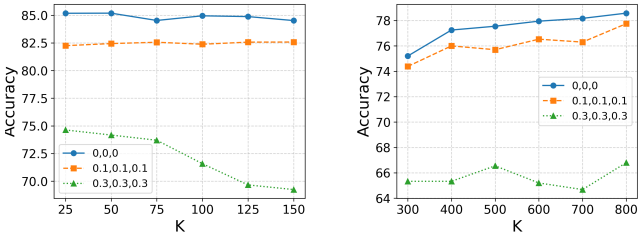
Table 3: Accuracy w.r.t Different Noise Injection Methods

Noise Type	Dataset	{gr,lr,fr}	GCN	Mid-GCN	RT-GNN	PRINGLE	RNCGLN	TRINA
Pair	Cora	{.1,.1,.1}	75.07±0.68	76.37±0.18	71.88±1.54	76.81±0.44	74.05±1.90	**82.38±0.19**
	CiteSeer	{.1,.1,.1}	**68.40±1.42**	62.75±0.72	59.06±1.40	66.87±0.83	65.85±1.19	**68.08±0.86**
	Photo	{.1,.1,.1}	88.58±0.32	74.10±0.49	85.65±0.35	90.73±0.12	78.81±0.74	**91.14±0.89**
	Pubmed	{.1,.1,.1}	68.25±0.53	70.57±0.64	67.49±0.87	62.23±0.57	69.10±0.65	**74.25±0.34**
	ogbn-arxiv	{.1,.1,.1}	40.20±0.11	26.28±0.07	42.16±0.06	51.32±2.66	OOM	**53.94±0.21**
Feature Based	Cora	{.1,.1,.1}	78.69±0.76	81.45±0.95	76.43±0.92	78.52±0.14	77.64±0.34	**83.38±0.23**
	CiteSeer	{.1,.1,.1}	69.23±0.75	68.48±0.64	63.15±1.53	69.23±0.60	68.78±0.31	**70.80±0.41**
	Photo	{.1,.1,.1}	90.07±0.28	73.66±0.03	86.36±0.17	**91.78±0.21**	76.77±0.88	91.38±0.30
	Pubmed	{.1,.1,.1}	70.18±0.33	69.70±1.53	66.61±1.56	62.77±0.09	66.73±2.16	**70.85±0.57**
	ogbn-arxiv	{.1,.1,.1}	50.22±0.14	35.14±0.37	48.71±0.08	**55.17±1.73**	OOM	52.24±0.18
Class Confusion	Cora	{.1,.1,.1}	74.91±0.46	75.99±0.74	74.51±0.30	75.76±0.60	70.15±0.62	**79.39±0.21**
	CiteSeer	{.1,.1,.1}	65.68±1.93	61.92±0.84	57.85±3.14	64.73±0.76	59.18±1.92	**66.63±0.80**
	Photo	{.1,.1,.1}	89.10±0.40	72.69±0.02	85.88±0.38	**90.43±0.37**	75.91±0.15	89.59±0.29
	Pubmed	{.1,.1,.1}	69.40±0.74	**69.90±0.76**	68.01±0.97	65.67±0.31	64.88±1.08	**69.58±0.44**
	ogbn-arxiv	{.1,.1,.1}	41.74±0.11	24.35±0.25	43.64±0.11	**53.14±0.23**	OOM	**52.92±0.25**

instead of the updated features X_t^1 in Eq. 1. As shown in Table 4, replacing the GCE loss with standard cross-entropy leads to a slight performance drop, confirming the robustness benefits of GCE. Removing either the SC or FC module causes significant degradation, highlighting their critical roles in noise suppression. The variant using X_o underperforms compared to TRINA, indicating that iterative feature updates contribute to more effective structure cleaning. Moreover, TRINA consistently outperforms TRINA_T, demonstrating the advantage of using multiple teachers to guide the student model under noisy conditions. Detailed results under varying noise rates are provided in the Appendix.

5.4 Hyperparameter Analysis

In this subsection, we investigate how the hyper-parameters K, λ_4 and $teacher_n$ affect the performance of TRINA. K represents the selection of the top-K nearest neighbors when initializing and enhancing the KNN-graph for the adjacency matrix A. λ_4 controls the weight of the KLD loss term in Equation 21. $Teacher_n$ controls the number of teacher models. To explore the parameter sensitivity, we alter K as {25, 50, 75, 100, 125, 150} for Cora, and {300, 400, 500, 600, 700, 800}, respectively. We alter λ_4 and $Teacher_n$ as {0.01, 0.1, 1, 10, 100, 1000} and {1, 2, 3, 4}. We report the results on the Cora and Pubmed graph corrupted by uniform noise. The experiments are conducted 4 times and the average results are shown in Figure 4, Figure 5, Figure 6. From the figures, we observe: For the K value, in small datasets such as Cora, a relatively small K can achieve the best results. When K increases, the performance of the model will deteriorate. This might be because selecting too many neighbors can introduce noisy edges. In larger datasets such as Pubmed, an appropriately larger K value needs to be set to obtain the best performance of the model. For the parameter λ_4, as the weight of KLD increases, the noise robustness of the model first increases and then decreases. This might be because if the weight of KLD is too small, it cannot exert the constraining effect of this loss function, while if the weight is too large, it will ignore other important constraining information. For the parameter $teacher_n$, the model performs better and better as n increases. This is in line with the intuition of the multi-teacher-single-student framework.

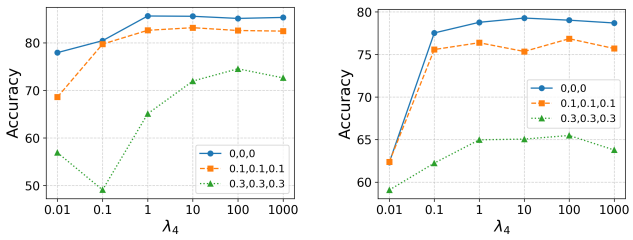


(a) Cora (b) Pubmed

Figure 4: The sensitivity of the hyper-parameter K,where the legend is the {gr,lr,fr}

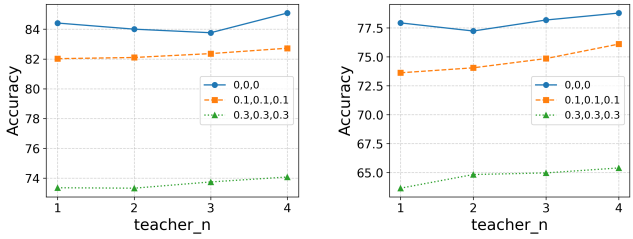
5.5 Efficiency Analysis (RQ4)

We conducted experiments to calculate the inference time for one sample on each dataset, using both the baseline method and TRINA.



(a) Cora (b) Pubmed

Figure 5: The sensitivity of the hyper-parameter λ_4 ,where the legend is the {gr,lr,fr}



(a) Cora (b) Pubmed

Figure 6: The sensitivity of the hyper-parameter $teacher_n$, where the legend is the {gr,lr,fr}

In Figure 7, we show the time to predict for one node in the baseline and TRINA for node classification tasks. From the figure, we can observe that GCN has the fastest inference speed. Next is Mid-GCN, but its inference speed on the Photo dataset is rather slow. The inference times of PRINGLE, RT-GNN and TRINA are similar, because their predictors all use weighted GCN. The model with the slowest inference speed is RNCGLN. Thus, it can be seen that the inference speed of our proposed model is acceptable.

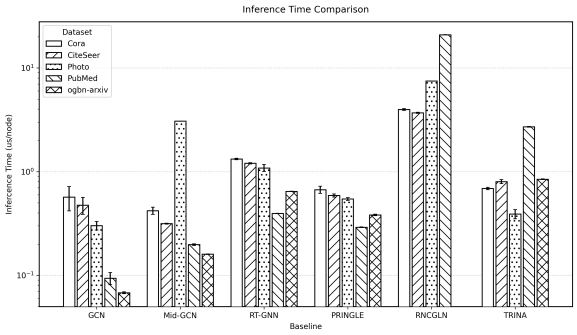


Figure 7: Inference time (us) comparison for a single node

6 Conclusion

We propose TRINA, a robust graph learning framework designed to address the challenging problem of trilateral noise in node features,

Table 4: Ablation Study

Noise Type	Dataset	{gr,lr,fr}	w/ CE	w/o SC	w/o FC	w/ X_o	TRINA_T	TRINA
Uniform	Cora	{.1,.1,.1}	80.55±0.26	76.26±0.88	80.96±0.35	81.30±0.42	80.30±1.08	**81.33±0.32**
	Pubmed	{.1,.1,.1}	75.58±0.15	67.95±1.18	72.83±0.55	74.03±0.11	73.95±1.56	**76.03±0.55**
Pair	Cora	{.1,.1,.1}	79.91±0.42	75.39±0.37	80.75±0.14	80.75±0.59	80.52±1.34	**82.38±0.19**
	Pubmed	{.1,.1,.1}	74.05±0.29	65.73±0.34	72.58±0.44	73.50±0.44	74.20±0.46	**74.72±0.70**
Feature Based	Cora	{.1,.1,.1}	82.12±0.30	80.36±0.49	82.66±0.29	83.00±0.30	82.33±0.15	**83.38±0.23**
	Pubmed	{.1,.1,.1}	71.55±0.66	**73.25±0.46**	67.53±0.51	69.20±0.39	70.90±0.53	71.71±0.70
Class Confusion	Cora	{.1,.1,.1}	**80.07±0.09**	75.29±0.51	79.35±0.26	79.05±0.75	78.39±0.38	79.39±0.32
	Pubmed	{.1,.1,.1}	69.50±0.46	60.50±0.42	68.48±0.55	69.35±1.06	69.68±1.36	**73.85±0.57**

graph structure, and labels. TRINA employs a two-stage learning strategy in which multiple teacher models generate reliable soft-label distributions and pseudo-labels to supervise a student model. This approach reduces the risk of error propagation commonly seen in previous methods. Furthermore, the SFC module iteratively refines both the graph topology and node features, enabling TRINA to adaptively suppress noise without assuming any part of the input is inherently clean. With Metis partitioning and adversarial subgraph embedding alignment, TRINA further scales to large graphs such as ogbn-arxiv while preserving robustness under trilateral noise. Extensive experiments on diverse datasets and noise types demonstrate that TRINA achieves SOTA performance and robustness. Ablation studies and efficiency analyses validate the effectiveness and practicality of each component.

References

[1] Yassine Abbahaddou, Sofiane Ennadir, Johannes F Lutzeyer, Michalis Vazirgiannis, and Henrik Boström. 2024. Bounding the Expected Robustness of Graph Neural Networks Subject to Node Feature Attacks. arXiv:2404.17947 https://arxiv.org/abs/2404.17947

[2] Haoran Chen, Xianchen Zhou, Jiwei Zhang, and Hongxia Wang. 2025. L2M-GCN: a New Framework for Learning Robust GCN against Structural Attacks. *Neurocomputing* 636 (2025), 129962.

[3] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-GCN: an Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 257–266.

[4] Enyan Dai, Charu Aggarwal, and Suhang Wang. 2021. NRGNN: Learning a Label Noise Resistant Graph Neural Network on Sparsely and Noisily Labeled Graphs. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 227–236.

[5] Enyan Dai, Wei Jin, Hui Liu, and Suhang Wang. 2022. Towards Robust Graph Neural Networks for Noisy Graphs with Sparse Labels. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 181–191.

[6] Yaroslav Ganin and Victor Lempitsky. 2015. Unsupervised Domain Adaptation by Backpropagation. In *International Conference on Machine Learning*. PMLR, 1180–1189.

[7] Zirui Guo, Lianghao Xia, Yanhua Yu, Yuling Wang, Zixuan Yang, Wei Wei, Liang Pang, Tat-Seng Chua, and Chao Huang. 2024. GraphEdit: Large Language Models for Graph Structure Learning. arXiv:2402.15183 https://arxiv.org/abs/2402.15183

[8] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems* 30 (2017).

[9] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang, and Jie Tang. 2022. GraphMAE: Self-supervised Masked Graph Autoencoders. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 594–604.

[10] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2021. Open Graph Benchmark: Datasets for Machine Learning on Graphs. arXiv:2005.00687 [cs.LG] https://arxiv.org/abs/2005.00687

[11] Jincheng Huang, Lun Du, Xu Chen, Qiang Fu, Shi Han, and Dongmei Zhang. 2023. Robust Mid-pass Filtering Graph Convolutional Networks. In *Proceedings of the ACM Web Conference 2023*. 328–338.

[12] Yao-Hui Huang, Jun-Wei Hsieh, Ming-Ching Chang, Lipeng Ke, Siwei Lyu, and Arpita Samanta Santra. 2021. Multi-Teacher Single-Student Visual Transformer with Multi-Level Attention for Face Spoofing Detection.. In *BMVC*. 125.

[13] Yeonjun In, Kanghoon Yoon, Kibum Kim, Sungchul Kim, and Chanyoung Park. 2024. Noise Robust Graph Learning under Feature-Dependent Graph-Noise.

[14] Wei Jin, Yao Ma, Xiaorui Liu, Xianfeng Tang, Suhang Wang, and Jiliang Tang. 2020. Graph Structure Learning for Robust Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 66–74.

[15] Wei Jin, Tong Zhao, Jiayuan Ding, Yozen Liu, Jiliang Tang, and Neil Shah. 2022. Empowering Graph Representation Learning with Test-time Graph Transformation. arXiv:2210.03561 https://arxiv.org/abs/2210.03561

[16] Hyeonsoo Jo, Fanchen Bu, and Kijung Shin. 2023. Robust Graph Clustering via Meta Weighting for Noisy Graphs. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1035–1044.

[17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[18] George Karypis and Vipin Kumar. 1998. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. *SIAM Journal on Scientific Computing* 20, 1 (1998), 359–392.

[19] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.

[20] Runlin Lei, Zhen Wang, Yaliang Li, Bolin Ding, and Zhewei Wei. 2022. EvenNet: Ignoring Odd-hop Neighbors Improves Robustness of Graph Neural Networks. *Advances in Neural Information Processing Systems* 35 (2022), 4694–4706.

[21] Kuan Li, Yang Liu, Xiang Ao, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. 2022. Reliable Representations Make a Stronger Defender: Unsupervised Structure Refinement for Robust Gnn. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 925–935.

[22] Wenjing Li, Jing Wang, Tingting Ren, Fang Li, Jun Zhang, and Zhongcheng Wu. 2022. Learning Accurate, Speedy, Lightweight CNNs via Instance-specific Multi-teacher Knowledge Distillation for Distracted Driver Posture Identification. *IEEE Transactions on Intelligent Transportation Systems* 23, 10 (2022), 17922–17935.

[23] Xianxian Li, Qiyu Li, Haodong Qian, Jinyan Wang, et al. 2024. Contrastive Learning of Graphs under Label Noise. *Neural Networks* 172 (2024), 106113.

[24] Yuwen Li, Miao Xiong, and Bryan Hooi. 2023. Graphcleaner: Detecting Mislabelled Samples in Popular Graph Learning Benchmarks. In *International Conference on Machine Learning*. PMLR, 20195–20209.

[25] Xiaorui Liu, Jiayuan Ding, Wei Jin, Han Xu, Yao Ma, Zitao Liu, and Jiliang Tang. 2021. Graph Neural Networks with Adaptive Residual. *Advances in Neural Information Processing Systems* 34 (2021), 9720–9733.

[26] Zhengyu Lu, Junbo Ma, Zongqian Wu, Bo Zhou, and Xiaofeng Zhu. 2024. A Noise-resistant Graph Neural Network by Semi-supervised Contrastive Learning. *Information Sciences* 658 (2024), 120001.

[27] Haodong Qian, Qiyu Li, Jinyan Wang, et al. 2024. RLGNN: Robust Graph Neural Networks with Noisy Label Learning. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*. IEEE, 1879–1886.

[28] Siyi Qian, Haochao Ying, Renjun Hu, Jingbo Zhou, Jintai Chen, Danny Z Chen, and Jian Wu. 2023. Robust Training of Graph Neural Networks via Noise Governance. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 607–615.

[29] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (2008), 93–93.

[30] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of Graph Neural Network Evaluation. arXiv:1811.05868 https://arxiv.org/abs/1811.05868
[31] Jaeyun Song, SungYub Kim, and Eunho Yang. 2023. RGE: a Repulsive Graph Rectification for Node Classification via Influence. In *International Conference on Machine Learning*. PMLR, 32331–32348.
[32] Yijun Tian, Chuxu Zhang, Zhichun Guo, Xiangliang Zhang, and Nitesh Chawla. 2022. Learning Mlps on Graphs: a Unified View of Effectiveness, Robustness, and Efficiency. In *The Eleventh International Conference on Learning Representations*.
[33] Daixin Wang, Jianbin Lin, Peng Cui, Quanhui Jia, Zhen Wang, Yanming Fang, Quan Yu, Jun Zhou, Shuang Yang, and Yuan Qi. 2019. A Semi-supervised Graph Attentive Network for Financial Fraud Detection. In *2019 IEEE International Conference on Data Mining (ICDM)*. IEEE, 598–607.
[34] Tailin Wu, Hongyu Ren, Pan Li, and Jure Leskovec. 2020. Graph Information Bottleneck. *Advances in Neural Information Processing Systems* 33 (2020), 20437–20448.
[35] Yuhao Wu, Jiangchao Yao, Xiaobo Xia, Jun Yu, Ruxin Wang, Bo Han, and Tongliang Liu. 2024. Mitigating Label Noise on Graph via Topological Sample Selection. arXiv:2403.01942 https://arxiv.org/abs/2403.01942
[36] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. 2020. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (2020), 4–24.
[37] Nan Yin, Li Shen, Chong Chen, Xian-Sheng Hua, and Xiao Luo. 2024. SPORT: a Subgraph Perspective on Graph Classification with Label Noise. *ACM Transactions on Knowledge Discovery from Data* 18, 9 (2024), 1–20.
[38] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2017. Spatio-temporal Graph Convolutional Networks: a Deep Learning Framework for Traffic Forecasting. arXiv:1709.04875 https://arxiv.org/abs/1709.04875
[39] Jingyang Yuan, Xiao Luo, Yifang Qin, Zhengyang Mao, Wei Ju, and Ming Zhang. 2023. ALEX: Towards Effective Graph Transfer Learning with Noisy Labels. In *Proceedings of the 31st ACM International Conference on Multimedia*. 3647–3656.
[40] Jingyang Yuan, Xiao Luo, Yifang Qin, Yusheng Zhao, Wei Ju, and Ming Zhang. 2023. Learning on Graphs under Label Noise. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
[41] Jinliang Yuan, Hualei Yu, Meng Cao, Jianqing Song, Junyuan Xie, and Chongjun Wang. 2023. Self-supervised Robust Graph Neural Networks against Noisy Graphs and Noisy Labels. *Applied Intelligence* 53, 21 (2023), 25154–25170.
[42] Zhilu Zhang and Mert Sabuncu. 2018. Generalized Cross Entropy Loss for Training Deep Neural Networks with Noisy Labels. *Advances in Neural Information Processing Systems* 31 (2018).
[43] Zhanke Zhou, Jiangchao Yao, Jiaxu Liu, Xiawei Guo, Quanming Yao, Li He, Liang Wang, Bo Zheng, and Bo Han. 2023. Combating Bilateral Edge Noise for Robust Link Prediction. *Advances in Neural Information Processing Systems* 36 (2023), 21368–21414.
[44] Yonghua Zhu, Lei Feng, Zhenyun Deng, Yang Chen, Robert Amor, and Michael Witbrock. 2024. Robust Node Classification on Graph Data with Graph and Label Noise. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17220–17227.
[45] Shuman Zhuang, Zhihao Wu, Zhaoliang Chen, Hong-Ning Dai, and Ximeng Liu. 2025. Refine Then Classify: Robust Graph Neural Networks with Reliable Neighborhood Contrastive Refinement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 13473–13482.

A Additional Experiments

A.1 Baselines

- GCN [19]: GCN is a popular graph convolutional network based on spectral theory.
- RNCGLN [44]: This method utilizes multi-hop graph and pseudo labels to separately handle graph structure noise and label noise.
- PRINGLE [13]:This method proposes the noise hypothesis FDGN, which believes that both the noise in the graph structure and the noise in the labels originate from feature noise. And based on this causal relationship, it handles the trilateral noise.
- RTGNN [28]:This method utilizes an edge classifier for graph augmentation, and uses the loss and confidence of labeled and unlabeled points to separately select clean samples and assign pseudo labels.
- Mid-GCN [11]:This method utilizes mid-frequency signals on graphs to filter out noisy graph structure noise.

A.2 Accuracy w.r.t Different Noise Injection Methods

Here, we present a comparison of the model performance under various noise ratios.The average performance and standard deviation over four independent runs are reported in Table 5. From the results, we make the following observations: (1) As the noise becomes increasingly severe, the performance of various methods also deteriorates more significantly. However, on all the noise ratios, TRINA achieved the best performance in almost every case. (2) Even at a high noise ratio, TRINA can still maintain relatively stable robustness, typically experiencing less performance degradation compared to other models. (3) Under the noise type of class confusion, the performance of each model declined the most, but TRINA still achieved relatively excellent results. However, on the Pubmed dataset, when the noise level was high, all models were subjected to very significant perturbation.

A.3 Ablation Study

To evaluate the effectiveness of each component in TRINA, we conduct an ablation study based on its single-teacher variant, denoted as "TRINA_T". Specifically, we assess the contribution of the GCE loss by replacing it with standard cross-entropy, resulting in the variant "TRINA_T w/ CE". We also evaluate the impact of the structure cleaning (SC) and feature cleaning (FC) modules by removing them individually, referred to as"TRINA_T w/o SC" and "TRINA_T w/o FC", respectively. In addition, we test the role of iterative feature refinement in structure cleaning by replacing the updated feature X_t^1 in Eq. 1 with the original noisy feature X_o, denoted as "TRINA_T w/ X_o". Detailed results under varying noise rates are provided in Table 6. From the results, we draw the following observations: (1) Under both pair noise and feature-based noise, under various levels of noise, each module of the model achieved the most significant results. (2) Under uniform noise, in most cases, each part of the model plays an important role, especially the SC, FC modules, and the use of X_t. (3) Under the Class confusion noise, at each noise ratio, each module of the model played a certain role.

Table 5: Accuracy w.r.t Different Noise Injection Methods

Noise Type	Dataset	{gr,lr,fr}	RNCGLN	PRINGLE	RT-GNN	Mid-GCN	GCN	TRINA
Pair	Cora	{0,0,0}	83.11±0.70	83.12±0.02	77.95±0.18	84.50±0.70	84.26±0.52	**85.35±0.35**
		{.1,.1,.1}	74.05±1.90	76.81±0.44	71.88±1.54	76.37±0.18	75.07±0.68	**82.38±0.19**
		{.3,.3,.3}	61.72±1.79	59.73±0.97	52.23±1.05	54.06±0.33	50.67±0.99	**70.89±0.86**
	CiteSeer	{0,0,0}	70.63±1.11	70.97±1.34	67.55±1.93	70.55±0.45	67.98±0.62	**72.35±0.32**
		{.1,.1,.1}	65.85±1.19	66.87±0.83	59.06±1.40	62.75±0.72	**68.40±1.42**	**68.08±0.86**
		{.3,.3,.3}	49.43±0.86	53.53±2.42	42.01±1.68	44.85±1.57	52.85±1.53	**55.05±1.63**
	Photo	{0,0,0}	87.16±0.62	**94.16±0.15**	88.97±0.35	81.96±0.06	92.36±0.13	**93.98±0.15**
		{.1,.1,.1}	78.81±0.74	90.73±0.12	85.65±0.35	74.10±0.49	88.58±0.32	**91.14±0.89**
		{.3,.3,.3}	58.52±1.47	81.27±0.18	73.55±0.30	57.23±0.91	78.27±0.60	**84.47±0.42**
	Pubmed	{0,0,0}	76.18±0.61	75.30±0.37	78.21±1.30	77.98±0.49	76.63±0.55	**78.93±0.27**
		{.1,.1,.1}	69.10±0.65	62.23±0.57	67.49±0.87	70.57±0.64	68.25±0.53	**74.25±0.34**
		{.3,.3,.3}	56.90±1.44	49.47±1.52	48.34±5.10	62.32±0.81	58.28±0.46	**63.25±0.34**
Feature Based	Cora	{0,0,0}	83.11±0.70	82.96±0.10	77.91±0.42	84.50±0.70	84.26±0.52	**85.47±0.11**
		{.1,.1,.1}	77.64±0.34	78.52±0.14	76.43±0.92	81.45±0.95	78.69±0.76	**83.38±0.23**
		{.3,.3,.3}	65.11±1.56	66.00±0.73	62.78±1.30	65.32±0.61	57.74±0.10	**75.54±0.37**
	CiteSeer	{0,0,0}	70.63±1.11	70.93±1.33	67.59±1.72	70.55±0.45	67.98±0.62	**72.25±0.18**
		{.1,.1,.1}	68.78±0.31	69.23±0.60	63.15±1.53	68.48±0.64	69.23±0.75	**70.80±0.41**
		{.3,.3,.3}	62.53±3.89	60.97±0.69	47.56±2.67	56.65±0.70	60.18±2.03	**62.97±1.36**
	Photo	{0,0,0}	87.16±0.62	**94.41±0.09**	89.07±0.44	81.97±0.07	92.36±0.13	93.89±0.15
		{.1,.1,.1}	76.77±0.88	**91.78±0.21**	86.36±0.17	73.66±0.03	90.07±0.28	**91.38±0.30**
		{.3,.3,.3}	57.62±2.64	86.35±0.32	78.83±1.56	58.64±0.80	84.95±0.66	**87.90±0.49**
	Pubmed	{0,0,0}	76.18±0.61	75.10±0.65	**78.25±1.24**	77.98±0.49	76.63±0.55	**78.95±0.09**
		{.1,.1,.1}	66.73±2.16	62.77±0.09	66.61±1.56	69.70±1.53	70.18±0.33	**70.85±0.57**
		{.3,.3,.3}	53.53±2.29	31.83±1.72	50.32±4.31	49.65±7.22	49.88±1.03	**64.35±0.88**
Class Confusion	Cora	{0,0,0}	83.11±0.70	83.10±0.04	77.77±0.22	84.50±0.70	84.26±0.52	**85.48±0.12**
		{.1,.1,.1}	70.15±0.62	75.76±0.60	74.51±0.30	75.99±0.74	74.91±0.46	**79.39±0.21**
		{.3,.3,.3}	48.12±1.12	56.20±0.61	54.05±1.08	59.57±0.76	60.64±0.37	**62.56±2.17**
	CiteSeer	{0,0,0}	70.63±1.11	70.93±1.33	67.71±1.75	70.55±0.45	67.98±0.62	**72.18±0.59**
		{.1,.1,.1}	59.18±1.92	64.73±0.76	57.85±3.14	61.92±0.84	65.68±1.93	**66.63±0.80**
		{.3,.3,.3}	33.95±1.93	45.50±0.75	39.99±1.22	42.62±1.32	48.93±1.92	**53.75±0.79**
	Photo	{0,0,0}	87.16±0.62	**94.30±0.08**	89.22±0.17	81.97±0.07	92.36±0.13	93.67±0.13
		{.1,.1,.1}	75.91±0.15	**90.43±0.37**	85.88±0.38	72.69±0.02	89.10±0.40	89.59±0.29
		{.3,.3,.3}	58.85±1.14	**80.16±0.39**	71.34±1.20	57.09±0.17	**80.23±0.21**	**78.70±1.47**
	Pubmed	{0,0,0}	76.18±0.61	75.47±0.50	78.23±1.32	77.98±0.49	76.63±0.55	**78.63±0.29**
		{.1,.1,.1}	64.88±1.08	65.67±0.31	68.01±0.97	**69.90±0.76**	69.40±0.74	**69.58±0.44**
		{.3,.3,.3}	42.50±1.90	41.87±2.60	**55.40±3.01**	**56.00±1.23**	53.48±0.90	46.85±0.63

Table 6: Ablation Study

Noise Type	Dataset	{gr,lr,fr}	TRINA_T w/ CE	TRINA_T w/o SC	TRINA_T w/o FC	TRINA_T w/ X_o	TRINA_T	TRINA
Uniform	Cora	{0,0,0}	82.87±0.29	79.28±0.27	83.28±0.50	83.32±0.37	83.16±0.44	**85.19±0.86**
		{.1,.1,.1}	81.26±0.46	71.76±0.77	79.54±0.57	80.72±0.36	**81.60±0.30**	81.33±0.32
		{.3,.3,.3}	72.84±1.40	60.93±0.69	67.32±0.56	71.71±0.80	**73.51±0.37**	73.18±1.00
	Pubmed	{0,0,0}	77.20±0.64	70.32±1.61	77.40±0.92	77.83±0.65	78.12±0.85	**78.78±0.43**
		{.1,.1,.1}	72.43±1.46	65.22±1.47	70.38±2.56	71.38±0.96	73.55±0.63	**76.10±0.24**
		{.3,.3,.3}	62.52±1.04	53.73±1.03	57.80±1.69	59.57±2.56	63.45±1.65	**65.40±0.51**
Pair	Cora	{0,0,0}	83.10±0.30	79.28±0.27	83.19±0.59	83.38±0.44	83.45±0.44	**85.35±0.35**
		{.1,.1,.1}	79.82±1.30	71.48±1.67	78.89±0.54	79.92±1.71	80.52±1.34	**82.38±0.19**
		{.3,.3,.3}	68.47±0.97	55.12±0.79	63.69±1.39	68.40±1.11	70.03±1.39	**70.89±0.86**
	Pubmed	{0,0,0}	77.20±0.70	70.32±1.61	77.60±0.67	77.53±0.75	77.75±0.54	**78.93±0.27**
		{.1,.1,.1}	68.90±1.14	62.95±1.27	70.50±0.76	70.62±0.84	70.80±1.86	**74.25±0.34**
		{.3,.3,.3}	58.40±1.38	56.48±1.11	56.08±1.68	57.15±1.39	61.73±1.61	**63.25±0.34**
Feature Based	Cora	{0,0,0}	82.74±0.36	79.28±0.27	83.15±0.32	83.19±0.35	83.29±0.56	**85.47±0.11**
		{.1,.1,.1}	81.46±0.54	74.99±0.39	80.07±0.62	81.76±0.30	82.33±0.15	**83.38±0.23**
		{.3,.3,.3}	73.24±1.54	66.01±1.07	67.42±0.67	71.20±1.06	73.26±0.42	**75.54±0.37**
	Pubmed	{0,0,0}	77.42±0.68	70.32±1.61	77.73±0.69	77.68±0.66	77.62±1.01	**78.95±0.09**
		{.1,.1,.1}	67.38±1.50	65.22±1.65	66.80±1.61	65.30±2.13	67.72±2.27	**70.85±0.57**
		{.3,.3,.3}	59.58±1.37	54.80±1.95	54.43±3.94	59.85±5.21	60.23±1.67	**64.35±0.88**
Class Confusion	Cora	{0,0,0}	82.44±0.56	79.28±0.27	83.00±0.30	83.07±0.49	83.25±0.62	**85.48±0.12**
		{.1,.1,.1}	77.49±0.74	68.85±2.07	77.24±0.70	76.67±1.07	75.92±0.91	**79.39±0.21**
		{.3,.3,.3}	**66.24±0.80**	60.83±1.10	56.91±2.51	61.80±1.28	65.15±0.43	62.56±2.17
	Pubmed	{0,0,0}	77.05±0.79	70.32±1.61	77.70±0.60	77.47±0.92	77.68±0.60	**78.63±0.29**
		{.1,.1,.1}	67.68±2.23	**70.32±1.61**	67.48±0.31	67.90±0.27	68.42±1.38	69.58±0.44
		{.3,.3,.3}	**47.52±3.31**	44.53±2.86	39.43±1.22	42.30±1.05	43.32±0.79	46.85±0.63